

## 1 Programmieren: Sprachen

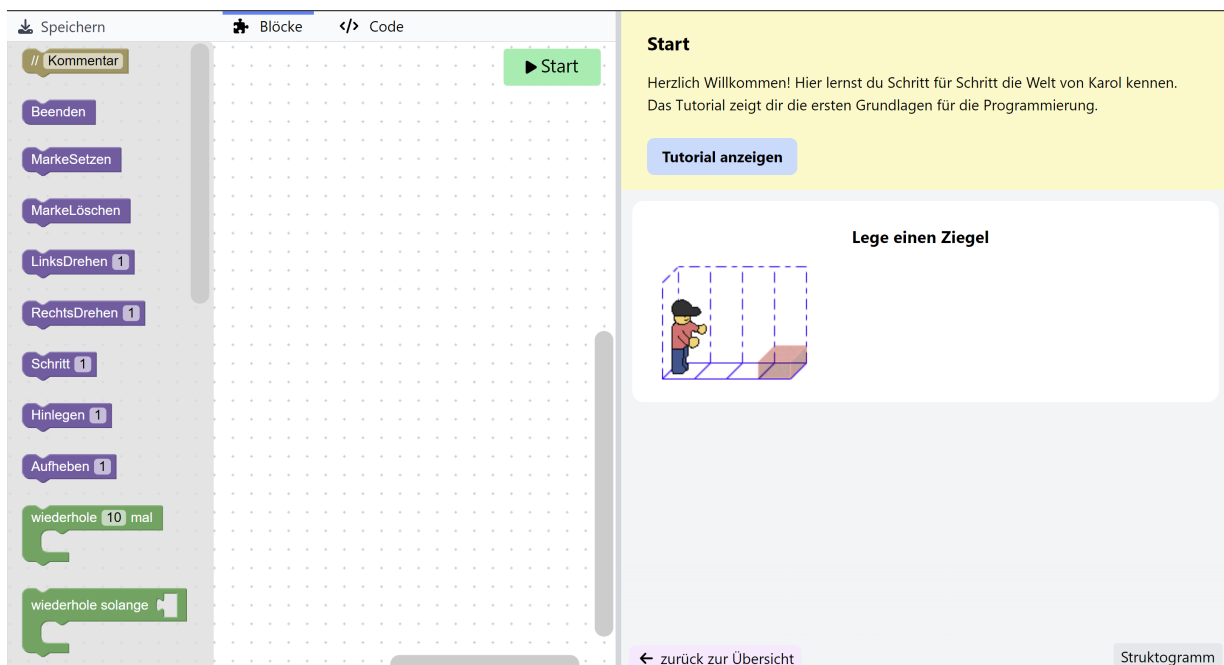


Beim Programmieren schreibt man präzise und unmissverständliche Befehle, die dann vom \_\_\_\_\_ in Maschinensprache übersetzt werden.

Hierfür gibt es verschiedene Programmiersprachen mit jeweils einer festgelegte **Grammatik und Rechtschreibung** (= \_\_\_\_\_). Dem gegenüber steht die **Logik eines Programms** (= \_\_\_\_\_), die weitgehend unabhängig von der Programmiersprache ist.

Bei den einsteigerfreundlichen **block-basierten Sprachen wie Blockly oder Online-Karol** übernehmen die Blöcke und ihre Verbindungsmöglichkeiten weitgehend die Einhaltung des Syntax. Bei **text-basierten Sprachen wie** \_\_\_\_\_ **oder** \_\_\_\_\_ müssen die Entwickler:innen selbst auf deren Einhaltung achten. Diese Sprachen bieten dadurch aber auch weitaus mehr Möglichkeiten (**man sagt auch: sie sind mächtiger**).

Dieses Skript arbeitet parallel mit den blockbasierten Sprachen Blockly und Online-Karol und der weit verbreiteten text-basierten und **objektorientierten** Sprache **Java**.



## Karol-Java Übersetzung

1. Bearbeite die Aufgaben von Robot Karol online: [karol.arrrg.de](http://karol.arrrg.de)
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

Schritt 2

MarkeSetzen

Schritt 10

wiederhole immer

LinksDrehen 1

wiederhole 10 mal

RechtsDrehen 4

wenn IstZiegel 1 dann

Aufheben 1

wenn Ist Norden dann

Schritt 1

sonst

LinksDrehen 1



## 2 Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Art

Online-Karol

Blockly

Java

Struktogramm

LinksDrehen 1

print " abc "

wenn dann  
sonst

if  
do  
else

wiederhole solange

repeat while  
do

wiederhole 10 mal

repeat 10 times  
do

Anweisung TueEtwas

to do something

TueEtwas

do something

// Kommentar

Kommentar

Konstruktor MeineKlasse  
do

new MeineKlasse

## Semikolon und geschweifte Klammern

Wann macht man in Java ein Semikolon und wann setzt man geschweifte Klammern ein?

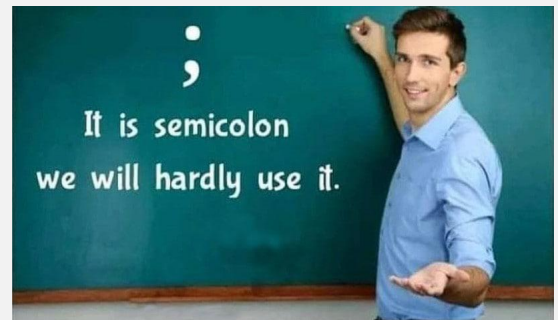
1. Überlege zuerst allein (2 Minuten)
2. Besprecht eure Überlegungen zu zweit und probiert sie in Online-Karol zur bestätigen (4 Minuten)

### 3 Strukturierung von Programmcode - Syntax

In Java wird Programmcode vor allem mit

strukturiert:

- \_\_\_\_\_ markieren das Ende eines **einfachen Befehls oder Methodenaufrufs** (normalerweise am Ende der Zeile).
- \_\_\_\_\_ markieren **Code-Abschnitte**, die wiederholt (Schleifen) oder bedingt (Bedingung) ausgeführt werden. Außerdem markieren sie den Methodenrumpf und Anfang und Ende einer Klasse. Je nachdem, was durch die Klammern markiert wird, spricht man synonym zu \_\_\_\_\_ der Klammern auch von \_\_\_\_\_ der Klasse, Methode, Schleife, etc.



**Computer Science Student :**



[www.reddit.com/r/ProgrammerHumor/comments/o0anud/semicolon](http://www.reddit.com/r/ProgrammerHumor/comments/o0anud/semicolon)

## 4 Strukturierung von Programmcode - Semantik: Methoden



Beim Programmieren können eigene Anweisungen/ Befehle/ Methoden/ Funktionen

**deklariert** werden. Bei der **Deklaration beschreibt** zunächst mit dem

\_\_\_\_\_, welche Eigenschaften die Methode hat und anschließend im

\_\_\_\_\_, wie genau sie sich verhalten soll. Eine deklarierte Methode

kann anschließend beliebig oft verwendet (=aufgerufen) werden.

Hierdurch kann man sehr lange Programme in übersichtlichere Einzelteile zerlegen, sodass

\_\_\_\_\_ reduziert wird, der Code durch aussagekräftige

\_\_\_\_\_ besser verstanden werden kann und Fehler schneller

gefunden werden.

In Java darf außerdem kein auszuführender Programmcode außerhalb von Methoden sein.

Die Deklaration besteht aus **Methodenkopf** und **Methodenrumpf** (von geschweiften

Klammern umschlossen):

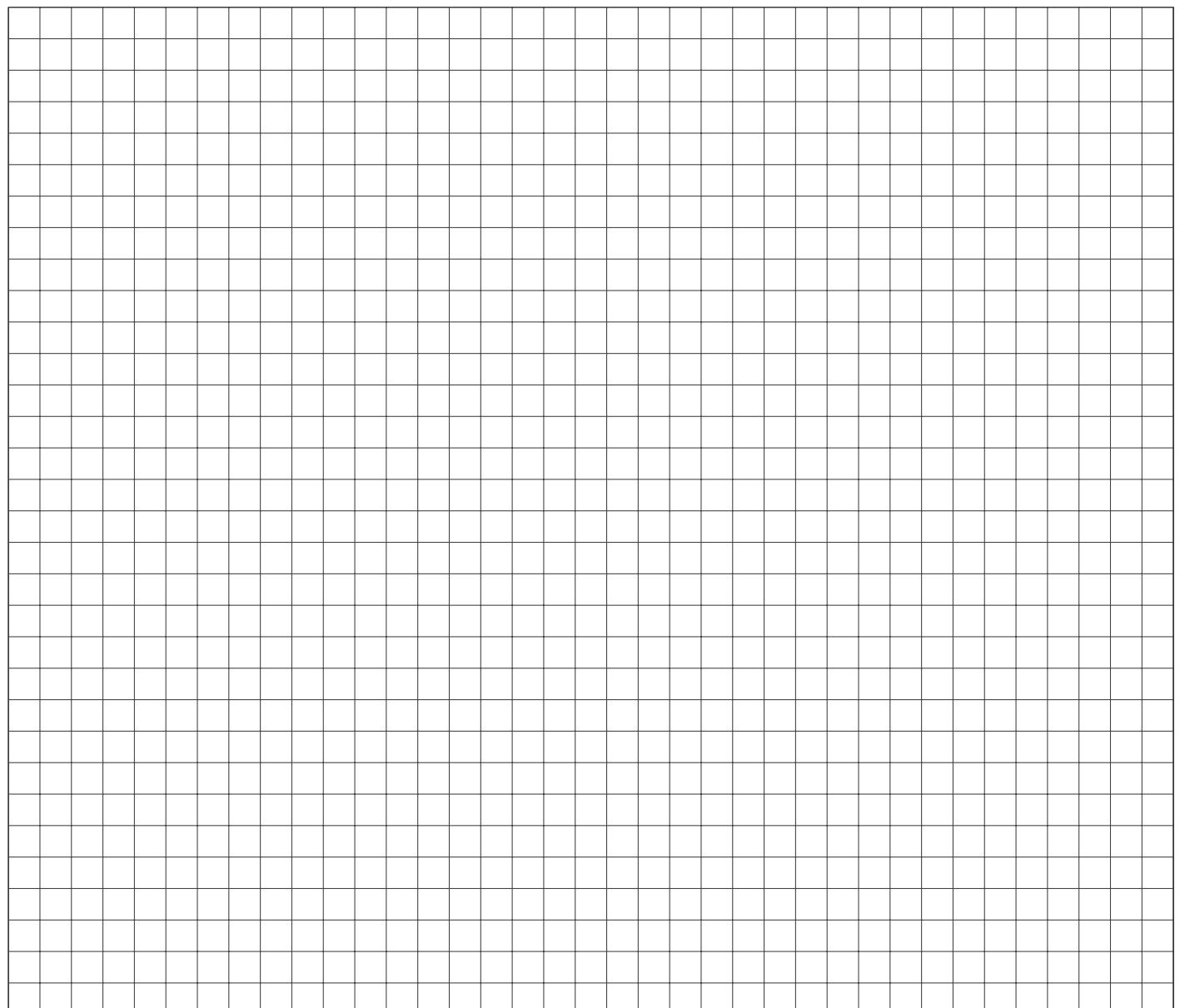
```
void tueEtwas() {  
    karol.schritt(2);  
}
```

Der **Aufruf** funktioniert wie ein einfacher Befehl:

```
tueEtwas();
```

 Übersetzungstraining: Methoden

Übersetze die folgenden Code-Schnipsel (**code snippets**) in Java-Code ohne den Computer zu verwenden.





## Karol mit Java und Struktogramme

- Bearbeite die Robot Karol ([karol.arrrg.de](http://karol.arrrg.de)) Level erneut. Schreibe den Programmcode dieses Mal direkt in Java! Zur Erinnerung: Deinen Code schreibst du an der gelb markierten Stelle in der main-Methode.
- Zeige nach den Schritten jeweils das Struktogramm an und notiere auf einem **Schmierzettel**, wie du die letzte Spalte in Hefteintrag 2 befüllen würdest.
- Was passiert, wenn du dein Programm aus eigenen Methoden aufbaust? Wieso ist das so?

```

1 class Programm {
2     Robot karol = new Robot();
3
4     void main() {
5                 
6     }
7 }

```

Lege einen Ziegel

← zurück zur Übersicht Struktogramm

## 5 Programmablauf grafisch darstellen: Struktogramme



Strukturgramme (auch: \_\_\_\_\_ )

stellen **Algorithmen (=Ablauf eines Programms)** grafisch dar. Ein Struktogramm bezieht sich immer auf genau eine Methode und stellt aufgerufene Methoden als ein Element dar.

In **Hefteintrag 2** sind die Struktogramm-Bausteine für verschiedene Befehlsarten dargestellt, die beliebig miteinander kombiniert werden können.

Zusätzlich gibt es folgenden Baustein, der eine leere Stelle (z.B. leerer else-Block) darstellt:

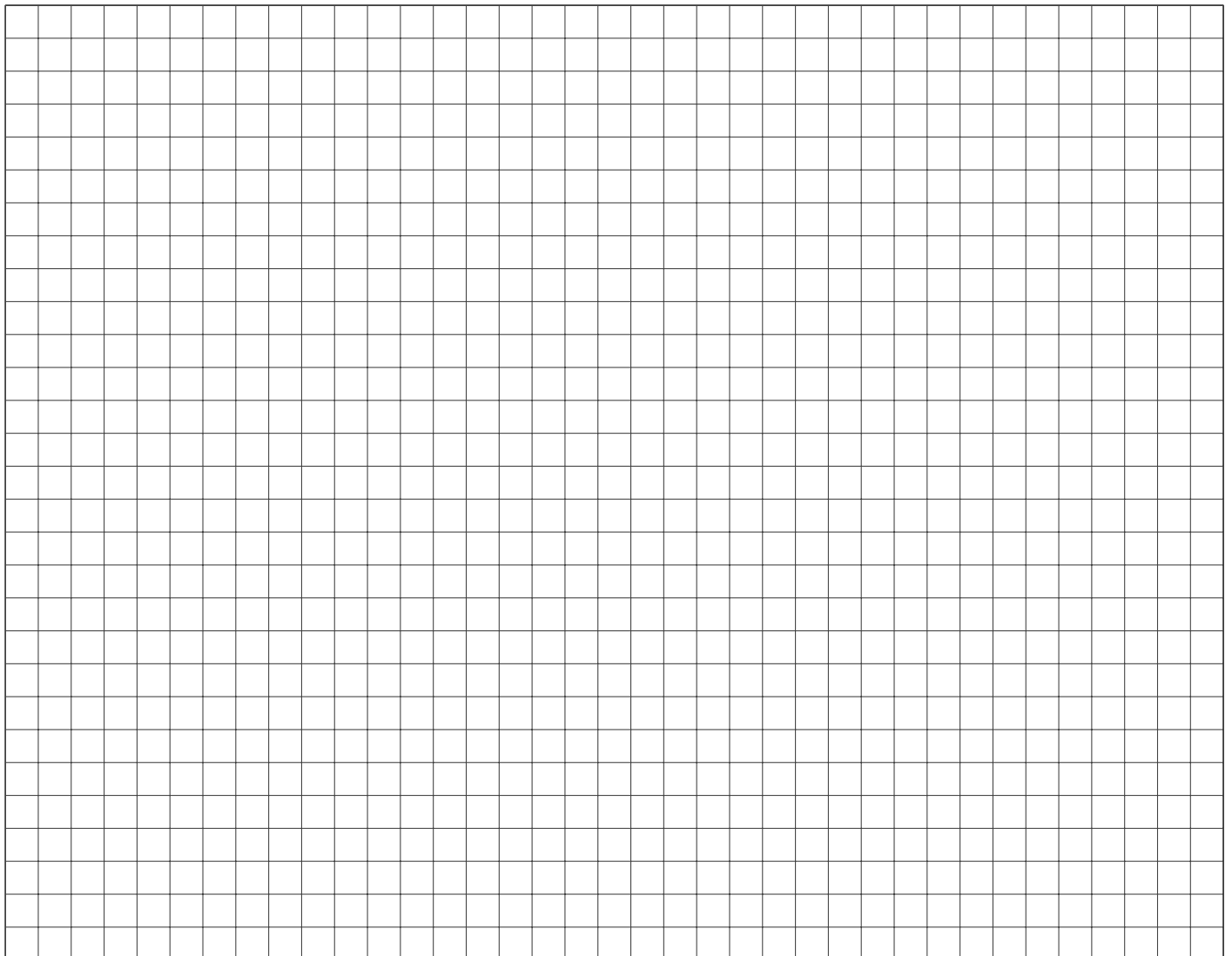
 **Struktogramm-Training**

Zeichne zu den folgenden Code Snippets jeweils ein Struktogramm:



```
ziegelImNordenLegen();  
quadratLaufen();  
tanzen();
```

```
void quadratLaufen() {  
    while (true) {  
        karol.schritt(10);  
        karol.linksDrehen();  
    }  
}
```





## 6 Taschenrechner

Implementiere das Klassendiagramm rechts mit Hilfe von Blockly. Probiere deine Methode aus (siehe Anleitung auf der Vorderseite). Fülle die Lücken im Java-Code unten aus, wenn deine Methoden funktionieren.

| Taschenrechner                                                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                   |
| + double addieren(double z1, double z2)<br>+ double subtrahieren(double z1, double z2)<br>+ double multiplizieren(double z1, double z2)<br>+ double dividieren(double z1, double z2)<br>+ double mittelwert(double z1, double z2) |

```
public class Taschenrechner
{
    public _____
        _____
}
public _____
    _____
}
public _____
    _____
}
public _____
    _____
}
}
```

## 7 Methodenköpfe



Im \_\_\_\_\_ können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

**Zugriffsmodifizierer** **Rückgabety**p **name**(**Datentyp** **parametername**){ //... }

**public void schritteLaufen**(**int anzahl**, ...) { //... }

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt:

---



---



---

Die verwendeten \_\_\_\_\_ haben folgende Bedeutungen:

**Zugriffsmodifizierer: public** - Methode kann ‚überall‘ verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

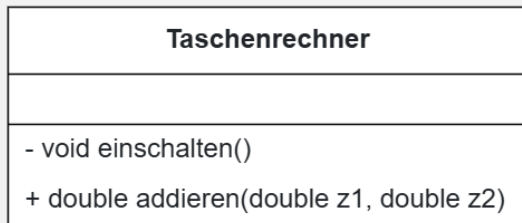
**Datentyp** - Art der Daten (z.B. Ganzzahlen: \_\_\_\_\_, Kommazahlen: \_\_\_\_\_, Text: \_\_\_\_\_, Wahrheitswert: \_\_\_\_\_)

**Rückgabety**p - Datentyp des Rückgabewerts (= \_\_\_\_\_). Kann z.B. Ergebnis einer Berechnung sein.

**Besonderer Rückgabety**p - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei **void** wird immer eine **return**-Anweisung benötigt.

**Parameter** - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

## 8 Methoden im Klassendiagramm



```
public class Taschenrechner {
    private void einschalten() {
        // ...
    }
    public double addieren(double z1, double z2) {
        return z1 + z2;
    }
}
```



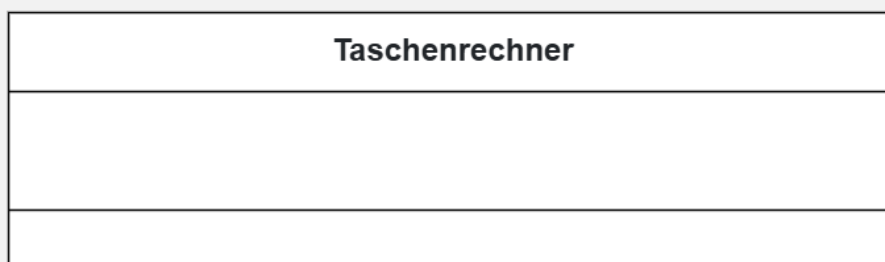
## Attribute im Klassendiagramm

Findet heraus, wie Attribute im Klassendiagramm dargestellt werden. Achtet auf **Zugriffsmodifizierer**, **Datentyp** und **Name**. Macht eine Skizze auf einem Schmierblatt.

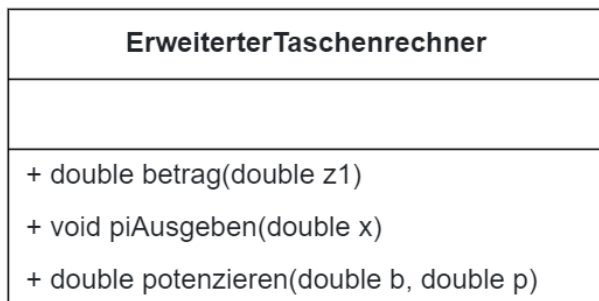
## 9 Attribute im Klassendiagramm



Attribute werden im Klassendiagramm folgendermaßen dargestellt:



### Klassendiagramm:



```
wiederhole x mal
    print(pi)
```

[illegible]

```
public class ErweiterterTaschenrechner
```

```
{
```

```
    public
```

```
}
```

```
    public
```

```
}
```

```
    public
```

```
}
```

```
}
```



## 10 Übersicht: Variablen

**Wertzuweisung** mit Gleichheitszeichen **von rechts nach links**.

Schema: **name = Wert;**

Beispiel: **alter = 14;**

**Verwendung des Werts** mit Variablenname statt eines Werts

Beispiel 1: **System.out.println(alter);** statt **System.out.println(14);**

Beispiel 2: **alter = alter + 1;** statt **alter = 14 + 1;**

**Vergleich des Werts** mit **< , <= , == , >= , > , != , name.equals(Wert)**

Beispiele: **x == 5** (x \_\_\_\_\_ 5) , **x != 5** (x \_\_\_\_\_ 5) ,

**text.equals("Hallo")** (text \_\_\_\_\_ ) ,

**Deklaration** = Erstellen einer Variable und Festlegung ihrer Eigenschaften.

**Arten von Variablen** in der objektorientierten Programmierung:

- \_\_\_\_\_ deklariert mit **Datentyp name;** in einem Code-Block und nur dort verfügbar.
- \_\_\_\_\_ (manchmal auch: Input/Argument) deklariert mit **(Datentyp name)** in den Klammern des Methodenkopfs und nur in dieser Methode verfügbar. Wert wird bei jedem Aufruf neu festgelegt.
- \_\_\_\_\_ deklariert am Anfang der Klasse mit **private Datentyp name;** Verfügbar in der ganzen Klasse mit anderem Wert für jedes Objekt. Zur Unterscheidung von lokalen Variablen kann bei der Verwendung auch **this.name** geschrieben werden.

## 11 Besondere Methoden: Getter und Setter



**Getter** haben den Zweck, den Wert eines (private) Attributs von anderen Klassen aus zugänglich zu machen. Der Methodenkopf folgt diesem Schema:

**public TypAttribut getNameAttribut()**

**Setter** ermöglichen das Setzen von Attributwerten von außerhalb derselben Klasse. Sie haben einen Parameter mit dem Datentyp des Attributs und Rückgabotyp void. Der Methodenkopf folgt diesem Schema:

**public void setNameAttribut(TypAttribut wert)**

**Standard-Getter/-Setter** lesen bzw. schreiben den Wert des Attributs direkt (ohne Prüfung). Allgemein können Setter vor dem Schreiben den Parameter auf Gültigkeit prüfen oder modifizieren.

| Spieler            |
|--------------------|
| - int x            |
| - int y            |
| - String imagePath |
|                    |

 **Spieler programmieren**

Probiere nach jedem Schritt aus, was passiert, wenn du dein Spiel startest!

1. Programmiere die Klasse Spieler wie im Klassendiagramm vorgegeben. Programmiere `getX()` und `getY()` als Standard-Getter, lasse `act()` erstmal leer.
2. Erstelle einen Konstruktor und setze dort die Koordinaten auf feste Werte und finde damit heraus, wie groß das Spielfeld ist. Hierfür musst du etwas herumprobieren.
3. Notiere den Programmcode von `getX()` und `getY()` auf der nächsten Seite.
4. Das Spielfeld führt die Methode `act()` seiner Spieler-Objekte ca. 25x pro Sekunde aus. Finde heraus, wie du damit ein Spieler-Objekt auf dem Spielfeld bewegen kannst.
5. Wie kann man die Bewegungsgeschwindigkeit des Spielers festlegen? Probiere deine Idee im Programm aus und ergänze das Klassendiagramm und den Programmcode auf der nächsten Seite.
6. Wie kann man zu Beginn des Spiels die Startposition des Spielers (bei jedem neuen Objekt woanders) festlegen? Probiere auch das wieder aus und ergänze das Klassendiagramm.
7. Über den Toolbox-Ordner 'Grafik: Objekte' kannst du in der Main weitere geometrische Formen hinzufügen. Nutze das, um ein kleines Spiel zu programmieren.

| Spieler                                            |
|----------------------------------------------------|
| - double x<br>- double y                           |
| + void act()<br>+ double getX()<br>+ double getY() |



```
public class Spieler {
    private double x;
    private double y;

    _____

    _____

    public double getX() {
        _____
    }
    public double getY() {
        _____
    }
    public void act() {
        _____
        _____
        _____
        _____
    }

    public void _____ {
        _____
    }

    public void _____ {
        _____
    }

    public _____ {
        _____
        _____
    }
}
```



## 12 Besondere Methoden: Konstruktoren

Konstruktoren sind spezielle Methoden, die genutzt werden, um ein **Objekt zu initialisieren/konstruieren**. Ein Konstruktor wird **automatisch beim Erstellen eines Objekts** aufgerufen.

In Java erkennt man einen Konstruktor insb. an zwei Dingen:

1. Er **heißt** \_\_\_\_\_ .
2. Er hat **keinen** \_\_\_\_\_ .

Ansonsten verhält er sich wie eine Methode und kann z.B. Parameter haben, die dann oft als Startwerte für das Objekt dienen.

Ein typischer Konstruktor im Programmcode sieht z.B. so aus:

```
public Spieler(double startX , double startY) {
    x = startX;
    y = startY;
}
```

Im Klassendiagramm würde dieses Beispiel so eingetragen:

Ein neues Objekt erstellt und speichert man dann z.B. so: **Spieler s1 = new Spieler(100,50);**



## 13 Besondere Methoden: Main/Hauptprogramm

Möchte man für ein Java-Programm starten, benötigt man eine \_\_\_\_\_ .

Diese nennt man auch den **Einstiegspunkt/Hauptprogramm** eines Programms. In ihr werden meistens die benötigten \_\_\_\_\_ erstellt und ihre \_\_\_\_\_ aufgerufen.

Der **Kopf** der Main-Methode ist immer gleich (siehe Bsp.):

```
public static void main() {
    Spieler s1 = new Spieler(100,50);
    // ... weitere Objekte und Methoden
}
```

Das Schlüsselwort \_\_\_\_\_ bedeutet hierbei, dass kein Objekt benötigt wird, um die Methode aufzurufen. In Blockly2Java wird die main-Methode automatisch zum Ausführen ausgewählt.